Report Load Testing Cargosync — Student 2. Roman Avanesov SV88/2024.

Endpoints:

For testing the customer order flow and warehouse management, we focused on the most critical endpoints.

Authentication:
- **POST /api/auth/register**
- **GET /api/auth/activate**
- **POST /api/auth/login**

Orders:
- **POST /api/orders**
- **GET /api/orders/{id}**

Warehouses:
- **GET /api/warehouses**
- **GET /api/warehouses/{id}**
- **GET /api/warehouses/{id}/availability**
- **GET /api/warehouses/{id}/sectors/{sectorId}/temperature-chart**

Products:
- **GET /api/products/catalogue**

Endpoint **POST /api/auth/login** is particularly interesting since it performs BCrypt password hashing on every request, offering a constant challenge to the backend regardless of caching. The temperature chart endpoint is the most demanding as it queries InfluxDB with Flux aggregation over 31.5 million data points. The product catalogue and warehouse list benefit from Redis caching, making them excellent candidates for high throughput testing.

Results (users/ramping):

TEST 10/2:

Type	Name	# reqs	# fails	Median	Average	Min	Max	req/s	50%	75%	90%	95%	99%
GET	/api/auth/activate	3	0	15	16	7	26	0.05	15	26	26	26	26
POST	/api/auth/login	11	0	97	108	78	271	0.18	97	110	110	270	270
POST	/api/auth/register	6	0	150	172	100	309	0.10	170	190	310	310	310
POST	/api/orders	10	0	75	230	46	1471	0.17	78	90	1500	1500	1500
GET	/api/orders/{id}	15	0	22	23	10	36	0.25	22	32	36	36	36
GET	/api/products/catalogue	70	0	18	60	7	590	1.18	18	31	230	280	590
GET	/api/warehouses	46	0	44	59	7	243	0.77	44	71	140	150	240
GET	/api/warehouses/{id}	28	0	32	48	9	462	0.47	35	45	66	77	460
GET	/api/warehouses/{id}/availability	2	0	13	24	13	35	0.03	35	35	35	35	35
GET	/api/warehouses/{id}/sectors/{sectorId}/temperature-chart	9	0	240	268	95	545	0.15	240	330	550	550	550
Aggregated		**204**	**0 (0%)**	**32**	**79**	**7**	**1471**	**3.43**	**33**	**89**	**190**	**290**	**570**

TEST 100/10:

Type	Name	# reqs	# fails	Median	Average	Min	Max	req/s	50%	75%	90%	95%	99%
GET	/api/auth/activate	65	0	22	282	3	9374	0.36	22	65	500	910	9400
POST	/api/auth/login	298	0	120	364	74	8600	1.66	120	280	910	1500	3300
POST	/api/auth/register	214	0	200	646	82	10935	1.19	200	590	2200	3000	3400
POST	/api/orders	255	0	170	660	17	7344	1.42	170	730	2000	3000	6500
GET	/api/orders/{id}	398	0	31	165	3	2944	2.21	31	120	540	740	2400
GET	/api/products/catalogue	2260	0	33	374	2	15025	12.57	33	240	860	2300	5200
GET	/api/warehouses	1551	0	26	254	2	10501	8.63	26	150	590	1300	3600
GET	/api/warehouses/{id}	842	0	26	239	2	9621	4.68	27	150	570	1100	3400
GET	/api/warehouses/{id}/availability	52	0	27	228	4	3491	0.29	28	99	770	1600	3500
GET	/api/warehouses/{id}/sectors/{sectorId}/temperature-chart	91	0	560	2932	5	14211	0.51	560	2500	11000	11000	14000
Aggregated		**6066**	**0 (0%)**	**44**	**366**	**2**	**15025**	**33.75**	**44**	**230**	**820**	**2000**	**5100**

TEST 1000/10:

Type	Name	# reqs	# fails	Median	Average	Min	Max	req/s	50%	75%	90%	95%	99%
GET	/api/auth/activate	282	11 (3.9%)	340	727	2	4321	2.35	340	1200	2200	2600	3600
POST	/api/auth/login	1127	288 (25.6%)	330	697	2	5159	9.40	330	950	1900	2400	3600
POST	/api/auth/register	793	0 (0%)	440	939	1	6227	6.61	440	1400	2500	3400	4700
POST	/api/orders	877	35 (4.0%)	430	984	2	8236	7.31	430	1600	2800	3500	4900
GET	/api/orders/{id}	1393	71 (5.1%)	310	947	1	7980	11.62	310	1500	2700	3800	5400
GET	/api/products/catalogue	7759	366 (4.7%)	410	1001	2	8283	64.70	410	1700	2900	3700	5300
GET	/api/warehouses	5374	243 (4.5%)	210	711	1	6619	44.81	210	1100	2200	2800	4100
GET	/api/warehouses/{id}	3022	158 (5.2%)	230	732	0	8378	25.20	230	1100	2200	2900	4600
GET	/api/warehouses/{id}/availability	218	10 (4.6%)	360	775	2	4480	1.82	370	1200	2500	2700	3900
GET	/api/warehouses/{id}/sectors/{sectorId}/temperature-chart	451	20 (4.4%)	460	978	2	7122	3.76	460	1500	2700	3200	5200
Aggregated		**21720**	**1270 (5.82%)**	**320**	**851**	**0**	**8378**	**181.1**	**320**	**1300**	**2500**	**3300**	**4800**

POST /api/auth/login

BCrypt hashing is inherently expensive and is the main bottleneck for login throughput. The 429 errors observed at 1000 users are expected and intentional — the rate limiter (10 req/min per IP in production, configurable) protects against brute-force attacks, which is a stated security requirement. In production, users come from different IPs so this does not affect legitimate traffic.

POST /api/auth/register

Registration involves BCrypt hashing and email sending (via SMTP). This is not a good candidate for caching, but could benefit from async email dispatch to reduce response time. Currently email is sent synchronously.

GET /api/products/catalogue

This endpoint is already optimized with Redis caching (catalogue cache, TTL 5 min). Performance at low load (18ms median at 10u) confirms the cache is effective. The RemoteDisconnected errors at 1000u are not specific to this endpoint — they are a general thread pool exhaustion issue.

POST /api/orders

Uses pessimistic locking (PESSIMISTIC_WRITE) on warehouse inventory rows to prevent overselling. This serializes concurrent order creation for the same products, which explains higher response times (430ms median at 1000u). A possible improvement would be optimistic locking with retry, or a queue-based order processing approach for very high throughput.

GET /api/warehouses and GET /api/warehouses/{id}

Both benefit from Redis caching (warehouseList TTL 30s, warehouseDetail TTL 30s). The 210–230ms median at 1000u is very good given the volume of concurrent requests and the 40,000 warehouse records in the database.

GET /api/warehouses/{id}/sectors/{sectorId}/temperature-chart

This is the most demanding endpoint, querying InfluxDB via Flux with aggregation over 31.5 million time-series data points. At 100u it shows 560ms median and 11,000ms at the 95th percentile. It benefits from the temperatureChart Redis cache (TTL 5 min), which explains why at 1000u the median actually improves to 460ms — many requests hit the cache. Adding a database index on the InfluxDB measurement tag warehouse_id and reducing the query window would further improve performance.

GET /api/warehouses/{id}/availability

Lightweight InfluxDB query (availability events only). Performance is consistently good across all load levels (13–370ms median). No significant improvements needed.

Conclusion:

From the results we can see that universally all endpoints started to fail at around 1000 users. There are several key factors to consider:

1. Tests are being run on a single machine. Specs are: macOS, Apple Silicon MacBook Pro M2 16gb
2. Running Docker with WSL-equivalent (Docker Desktop for Mac)

Machine is running macOS and using Docker Desktop, which adds significant overhead compared to native Linux. This limits the number of available worker threads and makes inter-container communication heavier.

The **5.82%** failure rate at 1000 users breaks down into two distinct categories. The first category is intentional failures — 276 HTTP **429 responses on /api/auth/login, which are produced by the rate limiter implemented as a brute-force protection requirement.** In a real deployment with users coming from different IP addresses, these errors would not occur. The second category is infrastructure failures — approximately 1000 RemoteDisconnected errors caused by Nginx dropping connections when the Tomcat thread pool is exhausted. The system does not crash under this load; it degrades gracefully by rejecting excess requests while continuing to serve the rest.

Redis caching proved highly effective: the product catalogue (TTL 5 min) and warehouse list (TTL 30s) maintain low response times even under extreme load, with GET /api/warehouses achieving a 210ms median at 1000 users across 40,000 records. **The most notable bottleneck is the temperature chart endpoint,** which requires a complex Flux aggregation query against 31.5 million InfluxDB data points. Even this endpoint benefits from the 5-minute Redis cache, explaining why its median response time actually improves from 560ms at 100 users to 460ms at 1000 users as cache hit rate increases.